Alpa

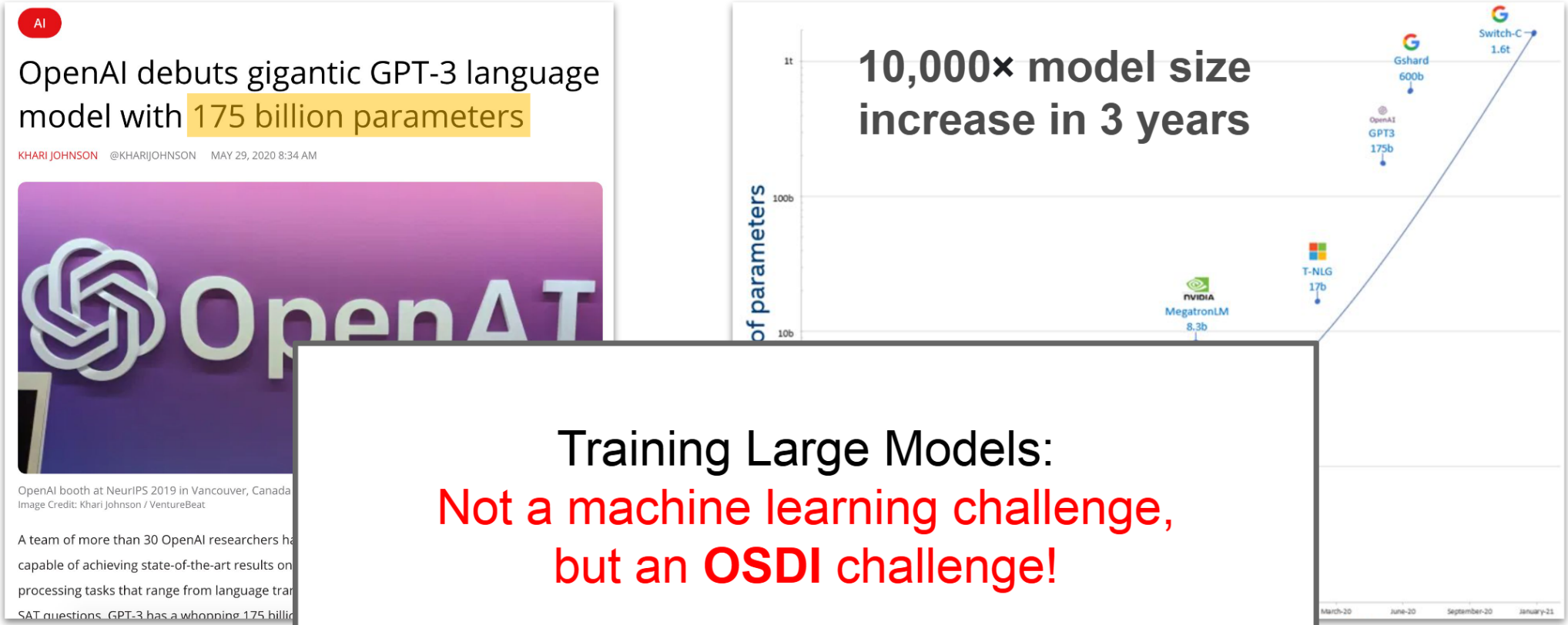
Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning

2024.01.14 이유찬

논문 정보

- 16th USENIX Symposium on Operating Systems Design and Implementation (USENIX OSDI 22)
- Lianmin Zheng, Zhuohan Li, and Hao Zhang, et al
- UC Berkeley, AWS, Google, Shanghai Jiao Tong Univ, Carnegie Mellon Univ, Duke Univ

Large Models Enable Breakthroughs in Machine Learning



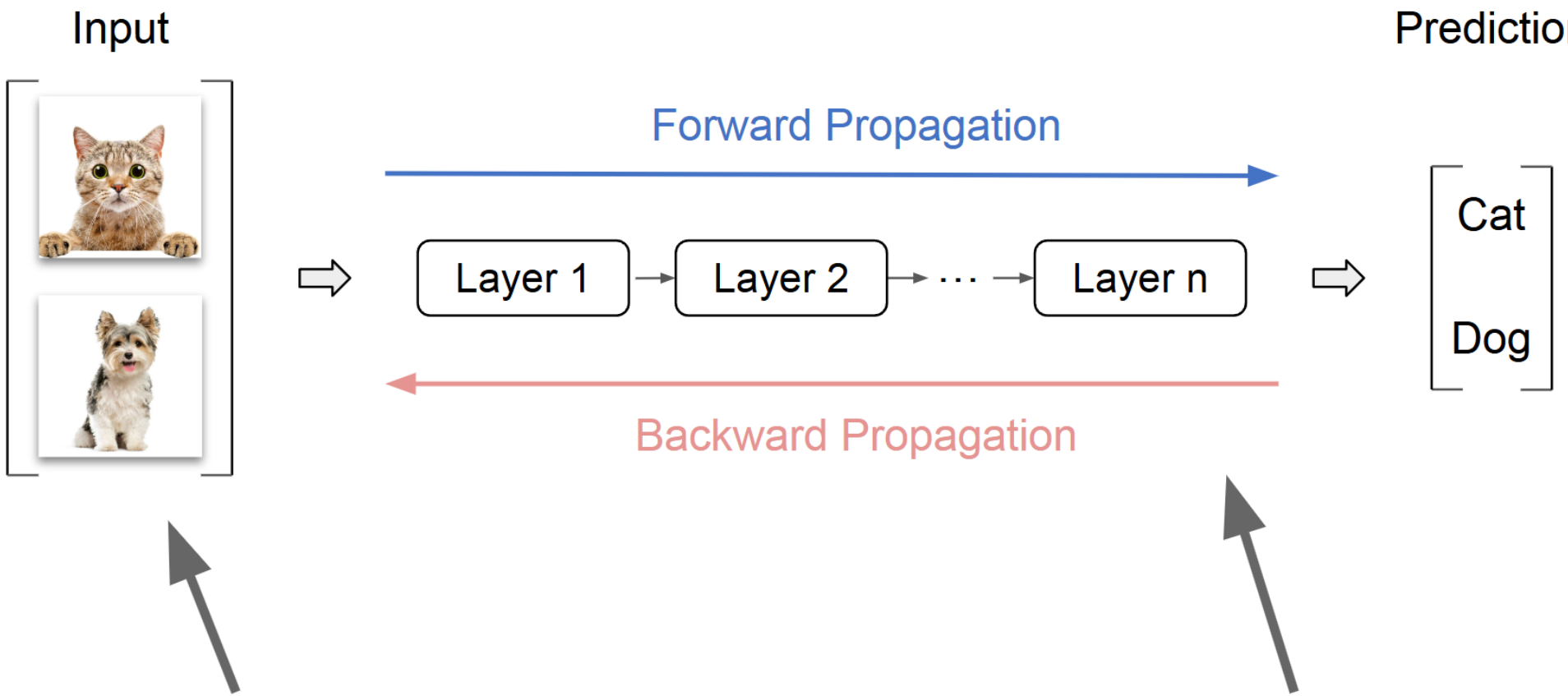
Training Large Models:

Not a machine learning challenge,

but an **OSDI** challenge!

Image source: towardsdatascience.com

Challenges

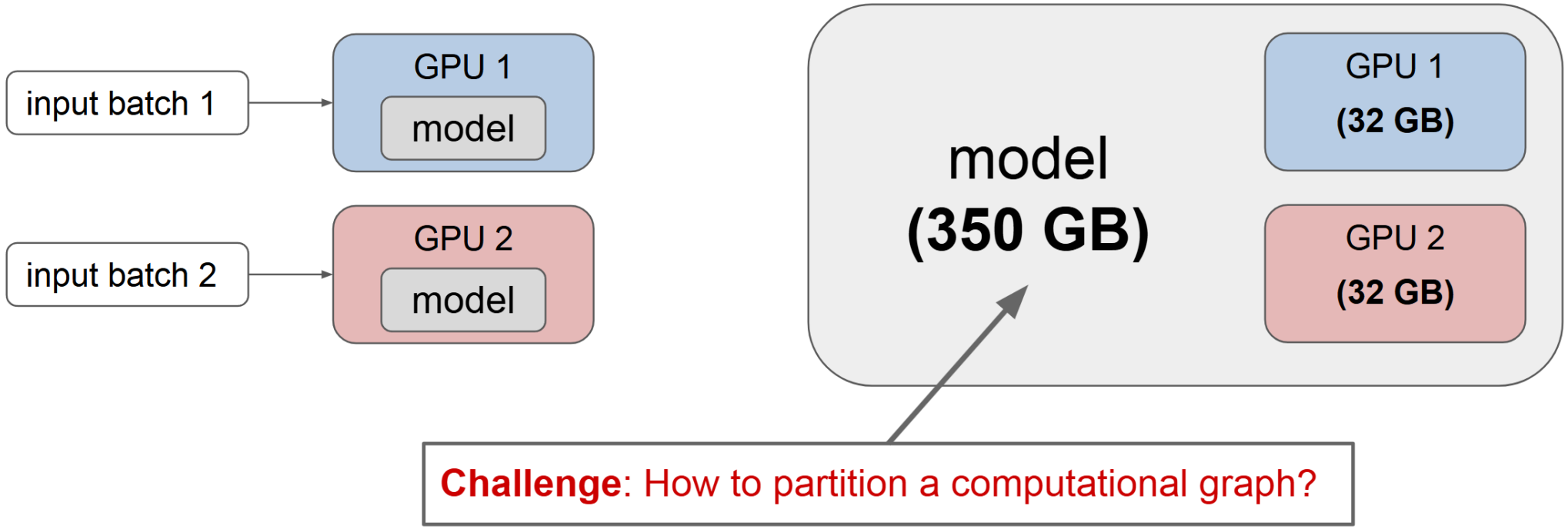


1. What if the **input dataset** is very large?

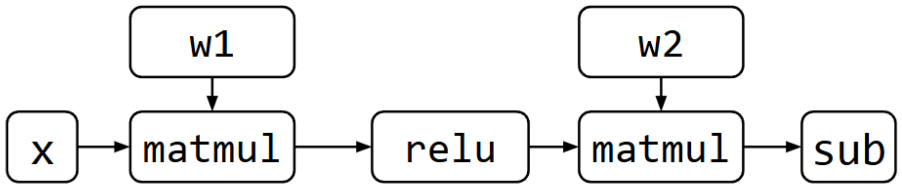
2. What if the **model** is very large?

Challenges

- Data가 큰경우
  - 비교적 쉬움
  - 모델을 복제하고 배치를 나눔
- 모델이 큰 경우
  - 비교적 어려움



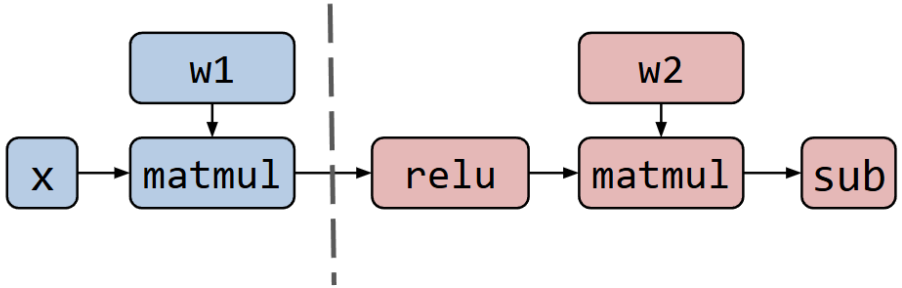
Partition Computational Graphs



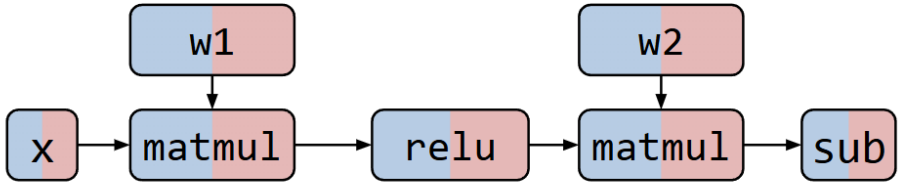
Device 1

Device 2

Strategy 1: Inter-operator Parallelism



Strategy 2: Intra-operator Parallelism

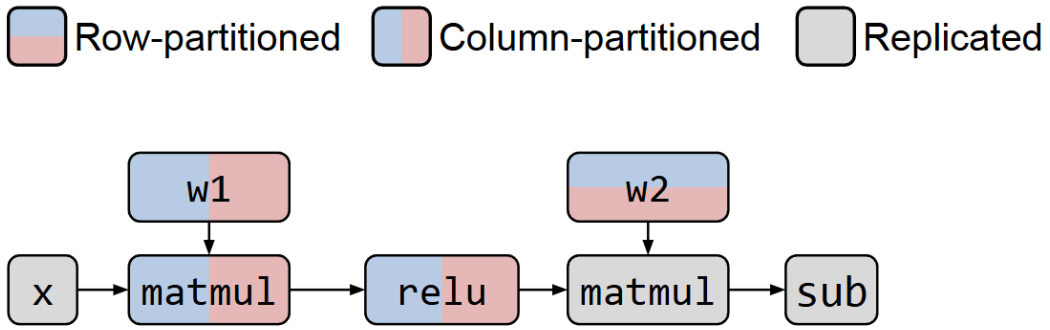


Trade-off

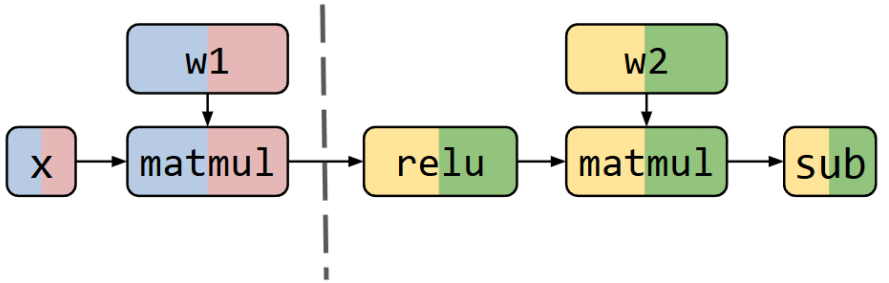
	Inter-operator Parallelism	Intra-operator Parallelism
Communication	Less	More
Device Idle Time	More	Less

Partition Computational Graphs

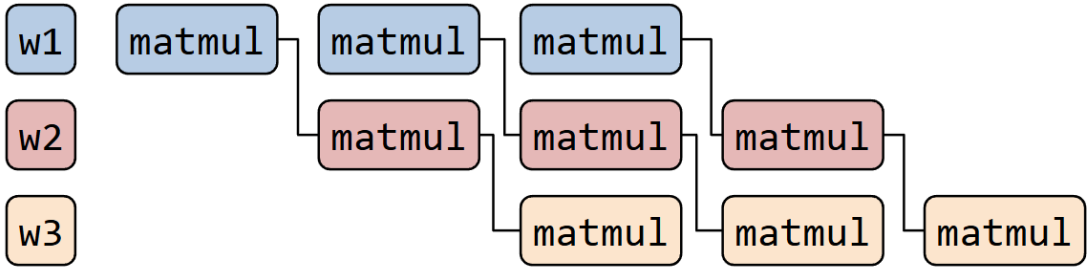
Multiple intra-op strategies for a single node

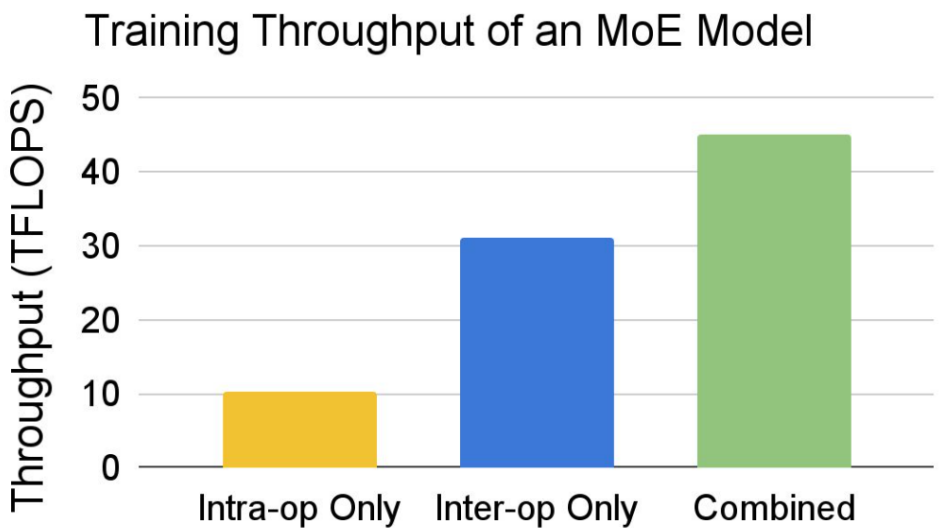


Combine Intra-op and Inter-op

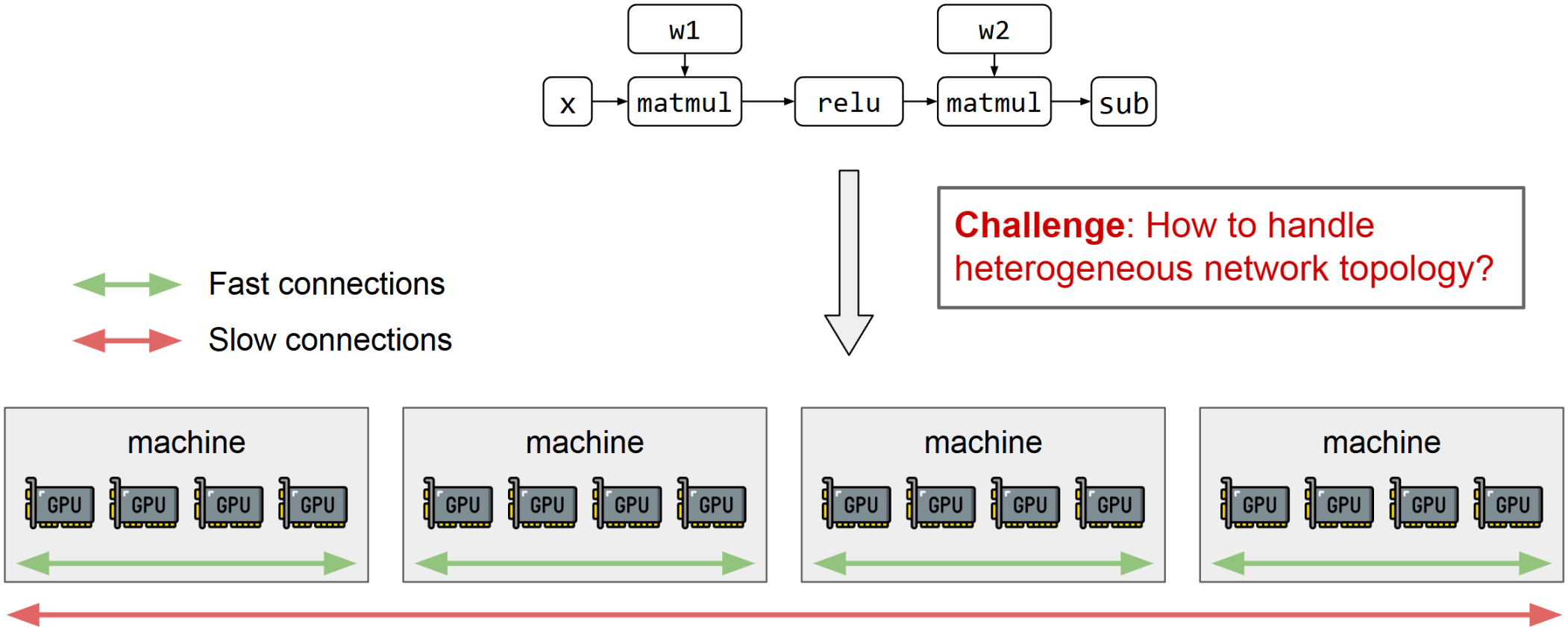


Pipeline the execution for inter-op parallelism

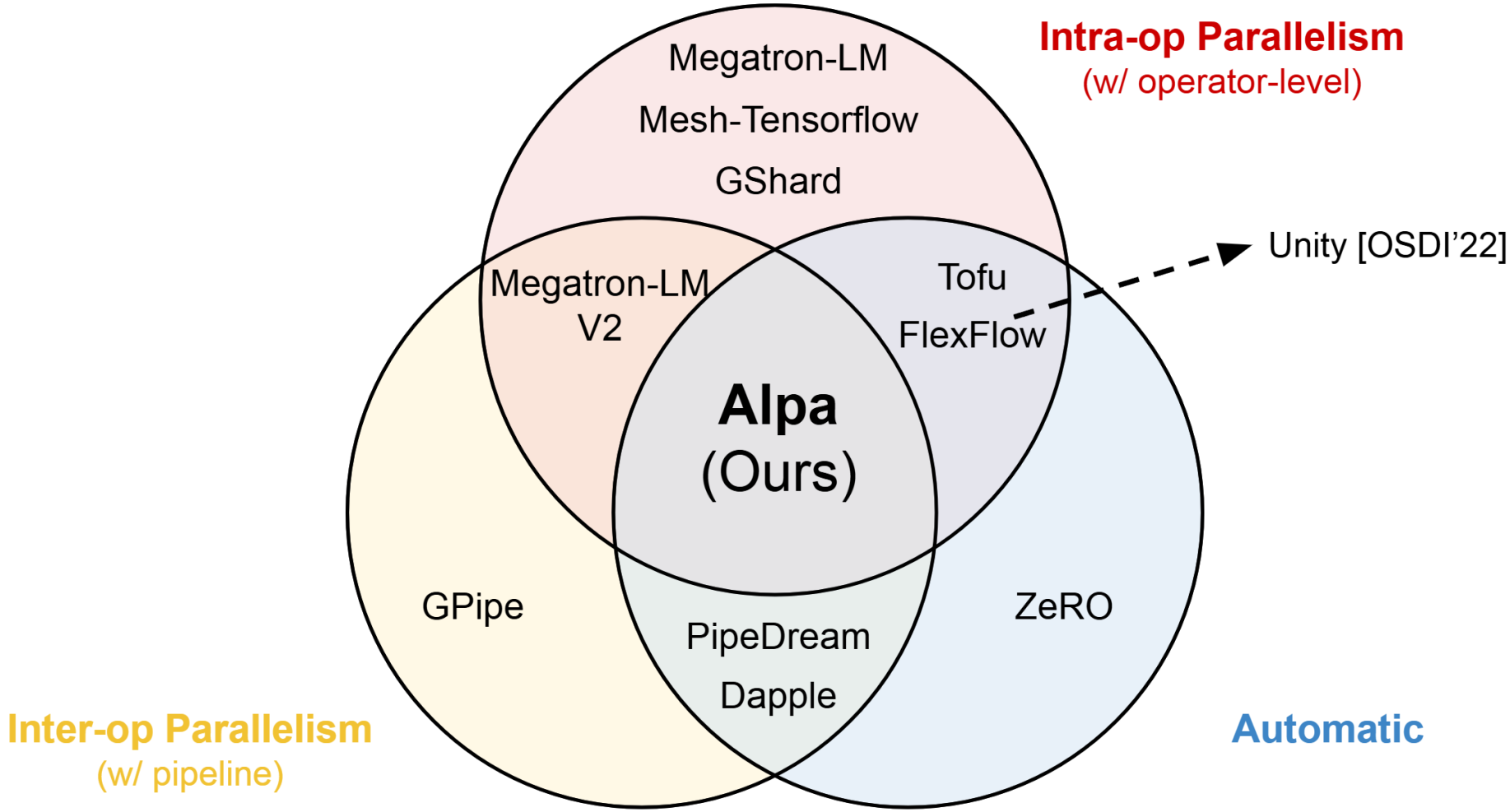




Heterogeneity



Prior Works


Intra-op Parallelism
(w/ operator-level)
Megatron-LM
Mesh-Tensorflow
GShard
Unity [OSDI'22]
Megatron-LM
V2
Tofu
FlexFlow
Alpa
(Ours)
GPipe
PipeDream
Dapple
ZeRO
Inter-op Parallelism
(w/ pipeline)
Automatic

Alpa Compiler

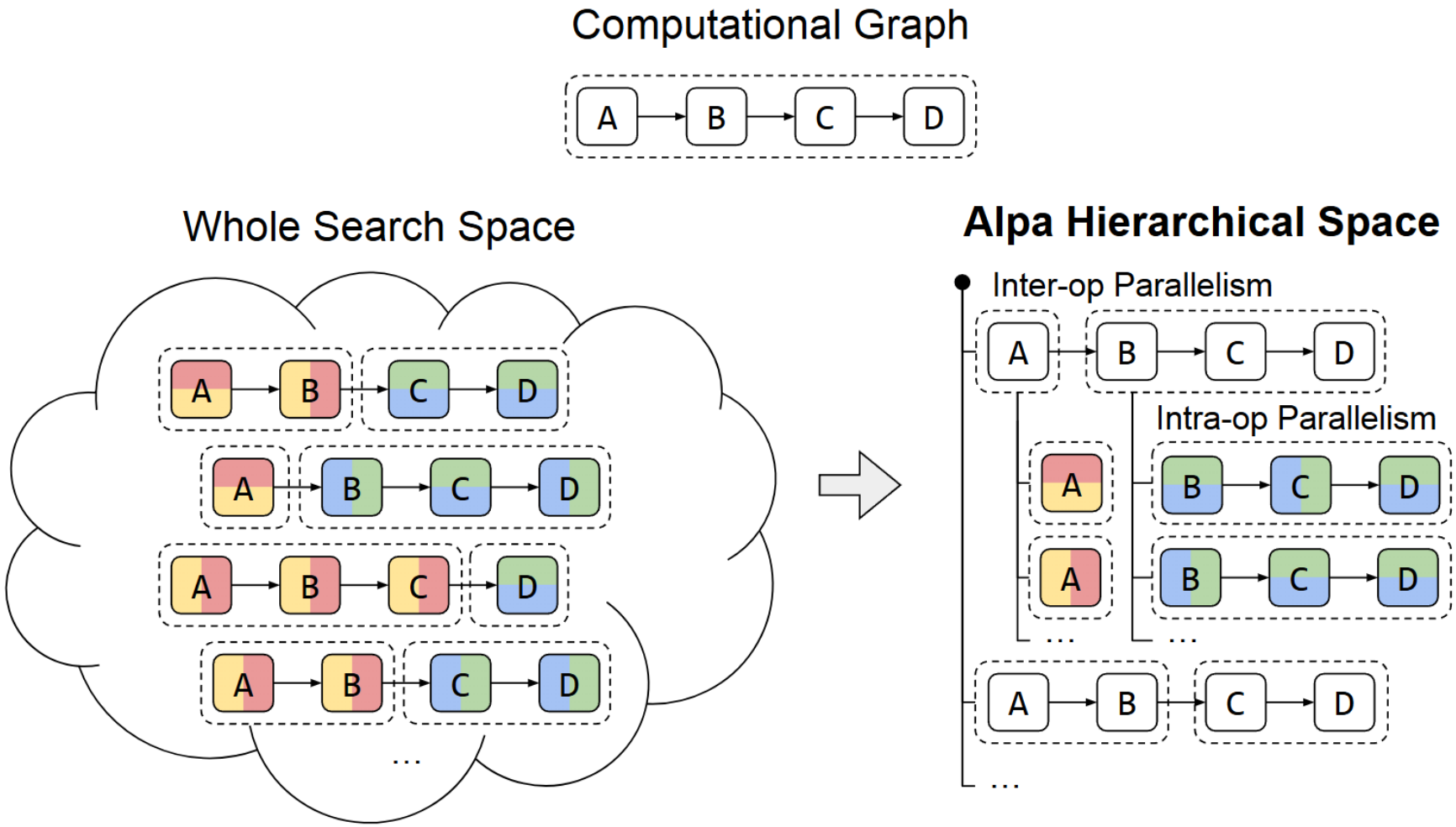
Contribution

Two-level hierarchical space of parallelism techniques.

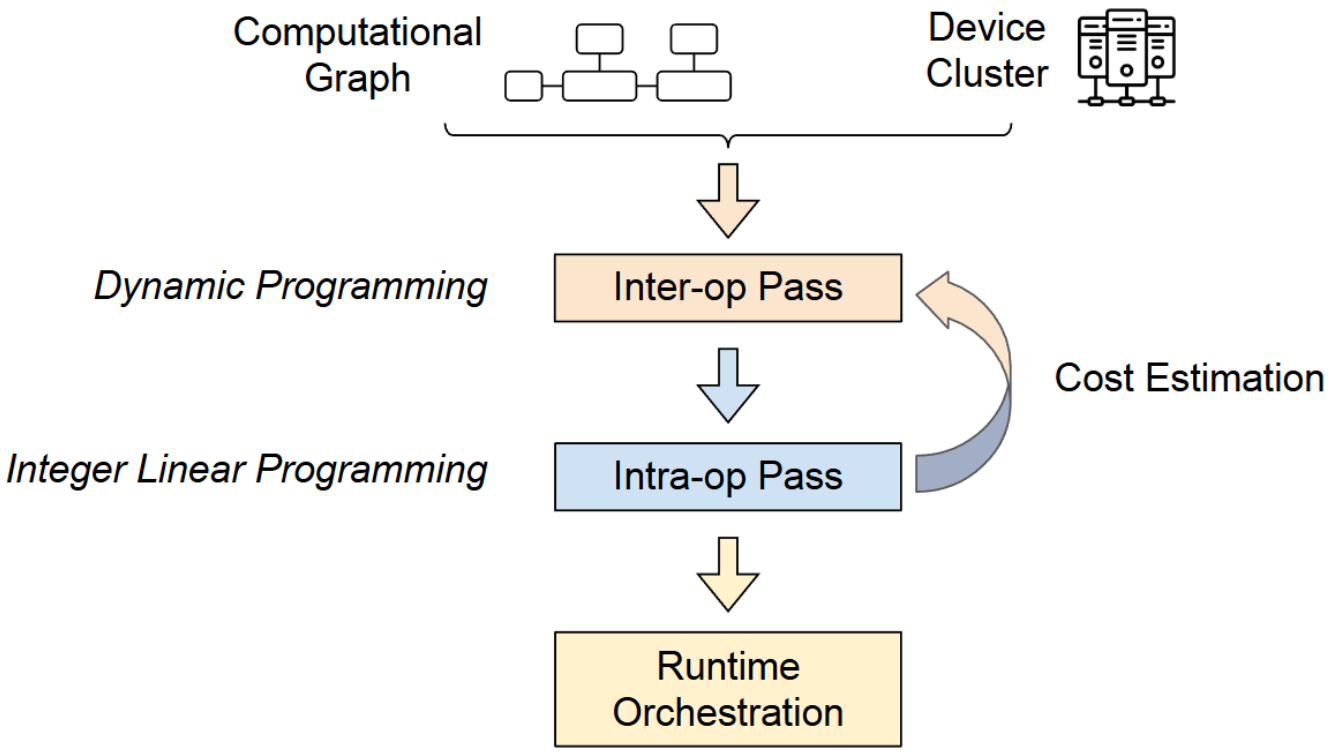
Effective optimization algorithms at each level.

Efficient compiler and runtime system implementation.

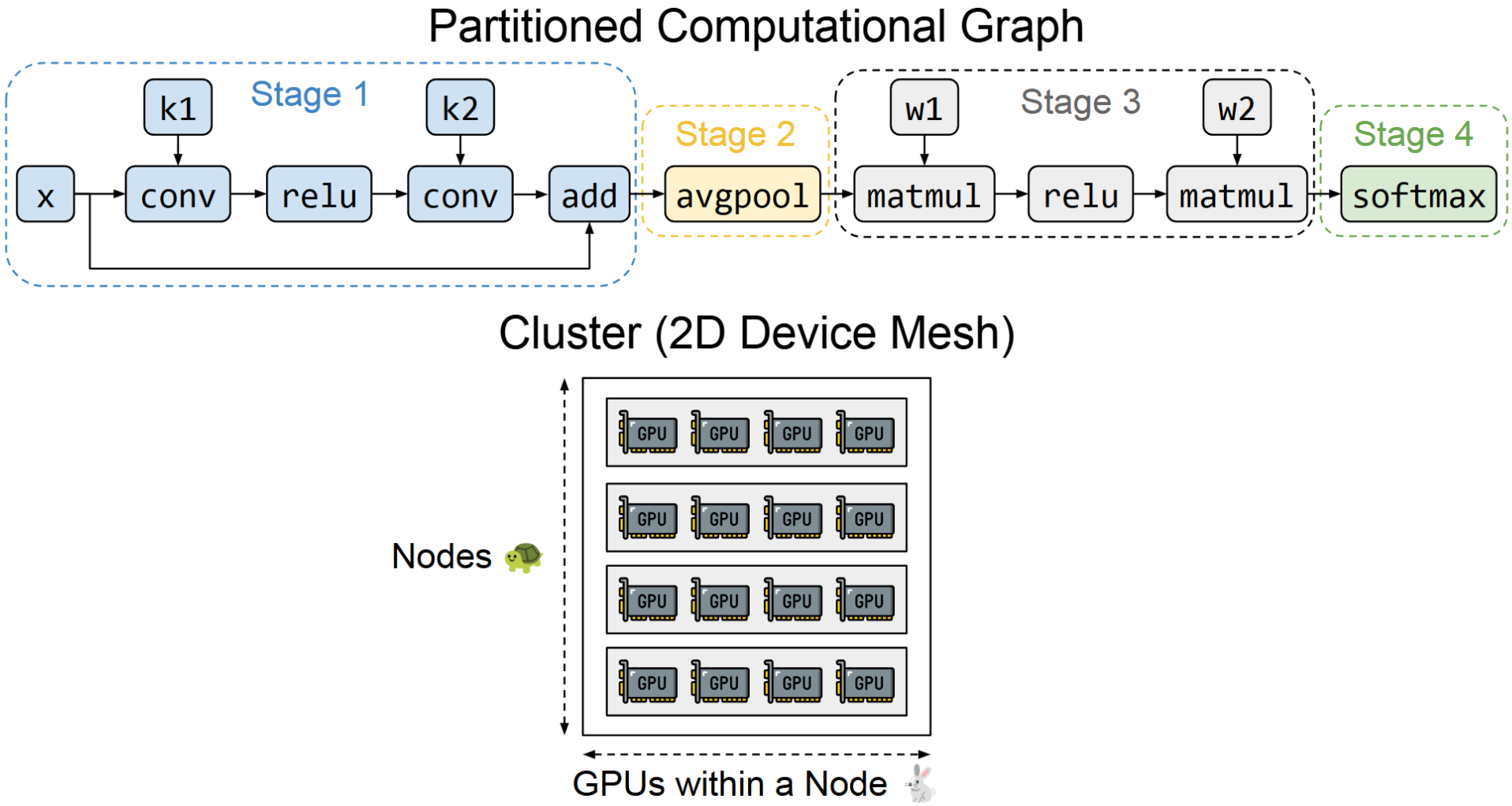
Optimization



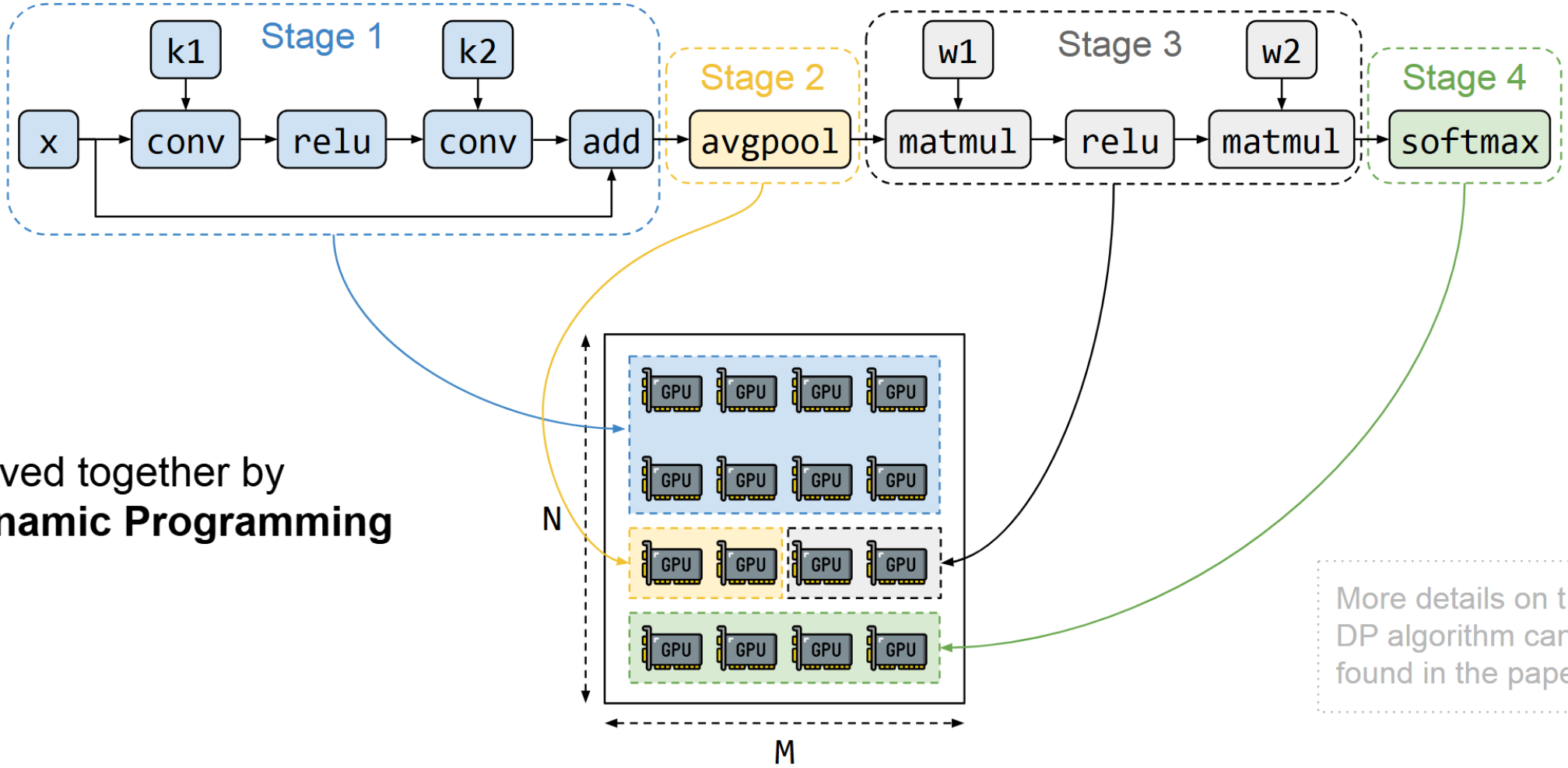
Optimization



Inter-op optimization



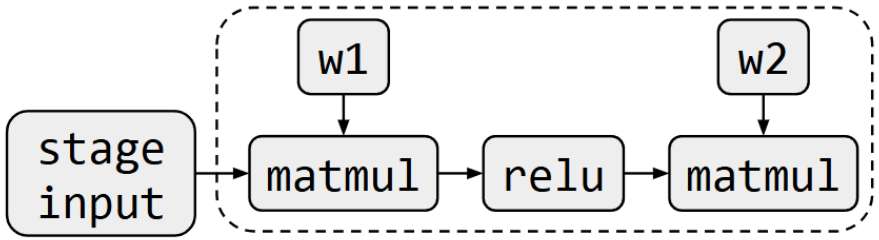
Inter-op optimization



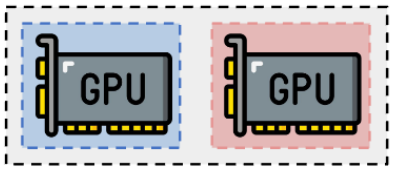
Solved together by
Dynamic Programming

More details on the DP algorithm can be found in the paper.

Intra-op optimization

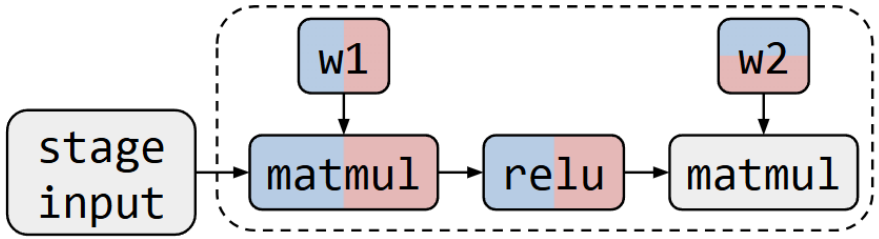


Stage



Submesh

Solved by
Integer Linear Programming



Stage with intra-operator parallelization

Intra-Op parallelism

Sharding spec

Table 1: Sharding specs of a 2-dimentional tensor on a 2×2 device mesh. A is a (N, M) tensor. The device mesh is [[Device 0, Device 1], [Device 2, Device 3]]. Each device stores a partition of A. The first column is the name of the sharding spec. The latter columns use Numpy syntax to describe the partitions stored on each device.

Spec	Device 0	Device 1	Device 2	Device 3
RR	$A[0:N, 0:M]$	$A[0:N, 0:M]$	$A[0:N, 0:M]$	$A[0:N, 0:M]$
S^0S^1	$A[0:\frac{N}{2}, 0:\frac{M}{2}]$	$A[0:\frac{N}{2}, \frac{M}{2}:M]$	$A[\frac{N}{2}:N, 0:\frac{M}{2}]$	$A[\frac{N}{2}:N, \frac{M}{2}:M]$
S^1S^0	$A[0:\frac{N}{2}, 0:\frac{M}{2}]$	$A[\frac{N}{2}:N, 0:\frac{M}{2}]$	$A[0:\frac{N}{2}, \frac{M}{2}:M]$	$A[\frac{N}{2}:N, \frac{M}{2}:M]$
S^0R	$A[0:\frac{N}{2}, 0:M]$	$A[0:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:N, 0:M]$	$A[\frac{N}{2}:N, 0:M]$
S^1R	$A[0:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:N, 0:M]$	$A[0:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:N, 0:M]$
RS^0	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:M]$	$A[0:N, \frac{M}{2}:M]$
RS^1	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:M]$	$A[0:N, 0:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:M]$
$S^{01}R$	$A[0:\frac{N}{4}, 0:M]$	$A[\frac{N}{4}:\frac{N}{2}, 0:M]$	$A[\frac{N}{2}:\frac{3N}{4}, 0:M]$	$A[\frac{3N}{4}:N, 0:M]$
RS^{01}	$A[0:N, 0:\frac{M}{4}]$	$A[0:N, \frac{M}{4}:\frac{M}{2}]$	$A[0:N, \frac{M}{2}:\frac{3M}{4}]$	$A[0:N, \frac{3M}{4}:M]$

Resharding spec

Table 2: Several cases of resharding. $all\text{-}gather(x, i)$ means an all-gather of x bytes along the i-th mesh axis. M is the size of the tensor. (n_0, n_1) is the mesh shape.

#	Src Spec	Dst Spec	Communication Cost
1	RR	S^0S^1	0
2	S^0R	RR	$all\text{-}gather(M, 0)$
3	S^0S^1	S^0R	$all\text{-}gather(\frac{M}{n_0}, 1)$
4	S^0R	RS^0	$all\text{-}to\text{-}all(\frac{M}{n_0}, 0)$
5	S^0S^1	$S^{01}R$	$all\text{-}to\text{-}all(\frac{M}{n_0 \cdot n_1}, 1)$

Intra-Op parallelism

- Parallel algorithms

Table 3: Several parallel algorithms for a batched matmul $C_{b,i,j} = \sum_k A_{b,i,k} B_{b,k,j}$. The notation $all\text{-}reduce(x, i)$ means an all-reduce of x bytes along the i-th mesh axis. M is the size of the output tensor. (n_0, n_1) is the mesh shape.

#	Parallel Mapping	Output Spec	Input Specs	Communication Cost
1	$i \to 0, j \to 1$	RS^0S^1	RS^0R, RRS^1	0
2	$i \to 0, k \to 1$	RS^0R	RS^0S^1, RS^1R	$all\text{-}reduce(\frac{M}{n_0}, 1)$
3	$j \to 0, k \to 1$	RRS^0	RRS^1, RS^1S^0	$all\text{-}reduce(\frac{M}{n_0}, 1)$
4	$b \to 0, i \to 1$	S^0S^1R	S^0S^1R, S^0RR	0
5	$b \to 0, k \to 1$	S^0RR	S^0RS^1, S^0S^1R	$all\text{-}reduce(\frac{M}{n_0}, 1)$
6	$i \to \{0, 1\}$	$RS^{01}R$	$RS^{01}R, RRR$	0
7	$k \to \{0, 1\}$	RRR	$RRS^{01}, RS^{01}R$	$all\text{-}reduce(M, \{0, 1\})$

- ILP(Integer Linear Programming)

$$\min_s \sum_{v \in V} s_v^T (c_v + d_v) + \sum_{(v,u) \in E} s_v^T R_{vu} s_u, \qquad (1)$$

s: parallel algorithm
d: communication cost
c: compute cost
R: resharding cost

C는 0으로 set

Inter-Op parallelism

- Pipeline parallelism

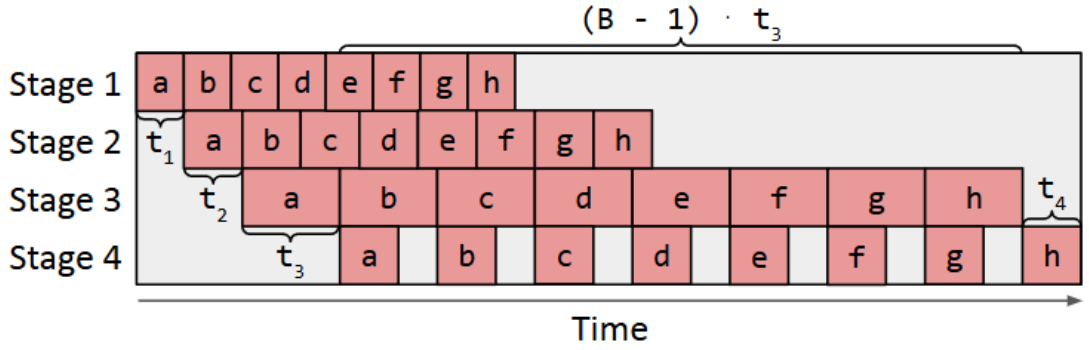


Figure 5: Illustration of the total latency of a pipeline, which is determined by two parts: the total latency of all stages ($t_1 + t_2 + t_3 + t_4$) and the latency of the slowest stage ($(B-1)\cdot t_3$).

- DP (Dynamic Programming)

$$T^* = \min_{\substack{s_1,\ldots,s_S;\\ (n_1,m_1),\ldots,(n_S,m_S)}} \left\{ \sum_{i=1}^{S} t_i + (B-1)\cdot \max_{1\le j\le S}\{t_j\} \right\}.$$

$$F(s,k,d;t_{max}) \qquad (3)$$
$$= \min_{\substack{k\le i\le K\\ n_s\cdot m_s\le d}} \left\{ \begin{array}{l} t_{intra}((o_k,\ldots,o_i),Mesh(n_s,m_s),s) \\ \quad + F(s-1,i+1,d-n_s\cdot m_s;t_{max}) \\ \mid t_{intra}((o_k,\ldots,o_i),Mesh(n_s,m_s),s) \le t_{max} \end{array} \right\},$$

$$G(k,r) \qquad (6)$$
$$= \min_{1\le i\le k} \left\{ \begin{array}{l} \max\{G(i-1,r-1),C(i,k)\} \\ \mid FLOP(o_i,\ldots,o_k) \le \dfrac{(1+\delta)FLOP_{total}}{L} \end{array} \right\},$$

Evaluation

Set up

- 8 nodes 64 GPUs
  - 8 NVIDIA V100 16GB
  - 64 vCPUs
  - 488GB memory
  - 노드 내 8 GPU는 NVLINK
  - 노드 간 연결은 25Gbps

Table 4: Models used in the end-to-end evaluation. LM = language model. IC = image classification.

Model	Task	Batch size	#params (billion)	Precision
GPT-3 [10]	LM	1024	0.35, 1.3, 2.6, 6.7, 15, 39	FP16
GShard MoE [31]	LM	1024	0.38, 1.3, 2.4, 10, 27, 70	FP16
Wide-ResNet [59]	IC	1536	0.25, 1.0, 2.0, 4.0, 6.7, 13	FP32

- 학습 처리량 비교 (PFLOPS)

- 모델 사이즈 scaling

Evaluation

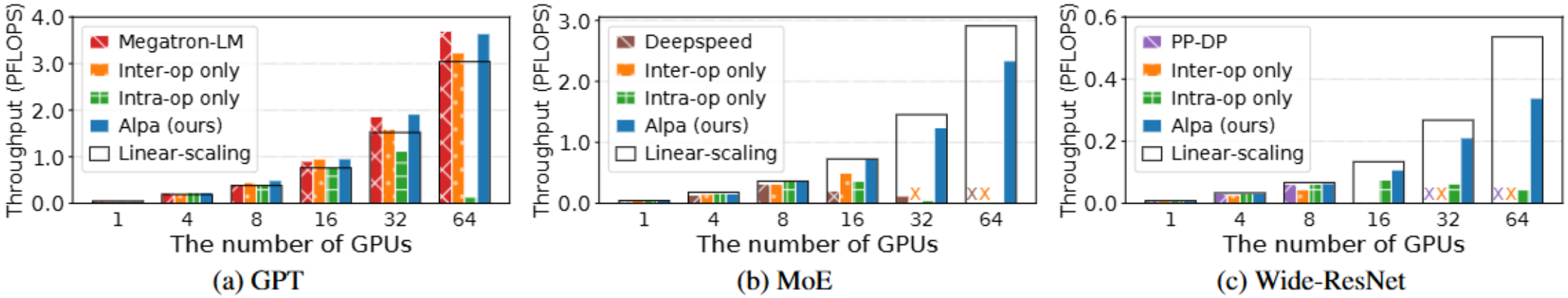
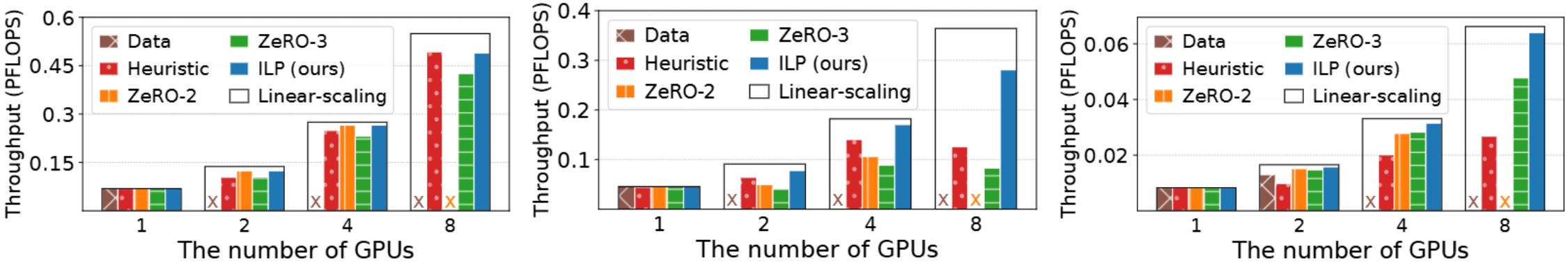


Figure 7: End-to-end evaluation results. "×" denotes out-of-memory. Black boxes represent linear scaling.

- GPT
  - Megatron과 유사한 성능
- MoE
  - Deepspeed는 inter-op만 사용

Evaluation



(a) GPT (b) MoE (c) Wide-ResNet

Figure 8: Intra-operator parallelism ablation study. "×" denotes out-of-memory. Black boxes represent linear scaling.